



中央财经大学

Central University of Finance and Economics

专业综合实践报告

组 长： 付德航 2022312151

组 员 1 谭 洁 2022312158

组 员 2 余锦帆 2022312141

日 期： 2025 年 12 月

分工及评分：

姓名	学号	工作内容	工作量	评分
付德航	2022312151	孪生网络的训练，Android 前端开发，pytorch 学习助手的搭建	35%	100
谭洁	2022312158	YOLOv11n 的训练，Android 前端开发，pytorch 学习助手的搭建	35%	100
余锦帆	2022312141	语义分割模型的数据标注和训练，pytorch 学习助手的资料整理	30%	100

目录

概述 3

任务一 信鸽虹膜对比与检索 4

 效果展示..... 4

 开始界面..... 4

 对比功能..... 5

 检索功能..... 6

快速开始.....10

具体实现.....11

 训练 Yolov11n 11

 数据预处理及训练..... 11

 评估 12

 获取眼部特写 13

 部署到手机端 14

 训练语义分割模型 14

 数据准备 14

 模型训练 15

 提取所有虹膜特征..... 15

 训练孪生网络 16

 产生训练数据..... 16

 评估与比较..... 17

前后端开发	17
数据库的构建	18
任务二 Pytorch 学习助手	18
效果展示	18
正常提问	18
查阅资料	19
运行代码	20
制定计划	21
具体实现	22
文档来源	22
工具实现	22

概述

本课程所有代码均可在 https://github.com/FDHTJ/professional_practice_course 找到，其中有些文件如原始数据集，训练好的孪生网络模型等文件太大，请访问 <https://pan.quark.cn/s/5b395b424cc0?pwd=hbML> 以获取。

打开网盘后，介绍如下：

images.zip	原始数据集图片，可解压缩到 backend/datasets/xinge_jpg/images
best_siamese_model_triple.pth	训练好的孪生网络模型 应放置在 backend/backend/main/siamese 下
pigeon_eye_segmentation_model.pth	训练好的语义分割模型，应放置在 backend/backend/main/segmentation 下
meta_data.json	从 details 中提取出并存储在向量数据库 中的元数据
chroma.sqlite3	生成的数据库，应放置在 backend/backend/database/pigeon_db 下

任务一 信鸽虹膜对比与检索

效果展示

开始界面



如图所示，开始界面中存在两个按钮，分别对应两个功能，一个是比对，即验证两只信鸽是否属于统一血统；二是检索，即对于给定的信鸽，检索与其相近的其他信鸽。

对比功能



进入对比页面后，首先点击两个图片的区域从本地上传图片，上传图片后，这里会自动调用训练好的 yolov11n 模型，将检测到的眼睛区域框出来，如果上传图片失败或者图片中并未检测到眼睛，这里都会弹出检测失败的提示，另外，如果一张图片中存在多个眼睛，那么只会选取置信度最高的那个进行比对。两张图片均上传好后，点击开始按钮便可以进行比对，届时检测到的图片区域会被上传到后端服务器，并返回检测结果。

检索功能



点击检索按钮以后，会来到检索页面，与对比页面同样的方式上传图片，点击开始检索稍等片刻后，便会返回检索内容（这里默认返回前 50 条）。



在这里上下翻动可以查看所有的检索结果，另外，如果这里鸽子图片加载失败，则会显示一张默认图片。点击对应的信鸽则可以跳转到该信鸽的详情页面。

16:40

4G

33



江苏金双强种(赛)鸽基地

CHN2007-22-338267雄

★★★★★

血统：国家一号X龙马力

父：B01-2243932

国家一号全兄弟X皇后号

母：B00-4011501

电脑龙马力199

00年江苏金双强公棚决赛15名

境外组冠军



石字：国家一号种鸽

ID：CHN07-22-338267

省份：江苏 城市：南京市

详细介绍

父：B01-2243932 血统：国家一号全兄弟X皇后号 祖父：B90-3127789 国家一号全兄弟，本身获杜尔登2734羽冠军、杜尔登919羽冠军、杜尔登1057羽亚军、杜尔登3547羽季军；

祖母：B92-2195263 贺伯特大名鸽皇后号，1992年比利时KBDB皇家赛鸽协会中距离鸽王全国冠军；1992年AVE REGINA鸽报鸽王全国冠军；1992年DE DUIVENKRANT鸽报鸽王全国冠军；1992年布拉帮省赛冠军。 母亲：B00-4011501 2000年江苏金双强公

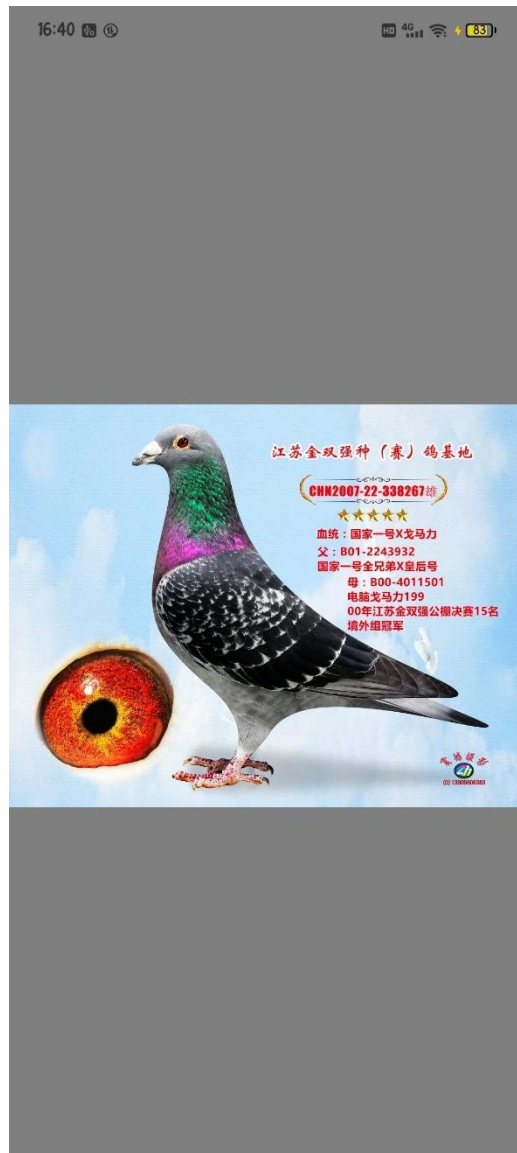
在这里可以查看到该信鸽的详情信息，包括其省份，以及详细介绍，如果觉得内容太长，可以点击旁边的小助手，让其帮你生成总结。



AI小助手总结

该信鸽为“国家一号种鸽”，ID CHN07-22-338267，产自江苏南京。性别雄性，羽毛呈灰黑底色带紫绿光泽，瞳孔橙红色。血统上，父为“国家一号全兄弟×皇后号”，母为“电脑戈马力199”（2000年江苏金双强公棚决赛15名境外组冠军）。祖先方面，祖父获杜尔登多赛事冠军，祖母获比利时及国内多项鸽王、省赛冠军。

在详情页面也可以点击图片，对图片进行放大查看。



而且，在此页面长按图片可以将图片保存到本地。注意，如果图片是默认图片，则无法进行放大查看以及保存 图片。

快速开始

如果只是想快速跑通代码查看效果，先将仓库克隆下载到本地，并根据 requirements.txt，配置好环境，之后将网盘中的数据按照概述中所说放置到固定的文件夹下，之后运行 backend/backend/main/Controller.py 文件，稍等片刻后，后端即可启动。

随后，可使用 Android Studio 打开项目 frontend/pigeon，这里使用的是局域网进行前后端的通信，因此，请注意将 frontend/pigeon/app/src/main/java/com/example/pigeon/utils/Constant.java 文件中

的 REQUEST_URL 换为本地的 ip 地址和端口号，另外，您需要自己在 <https://bigmodel.cn/console/> 申请一个 api key，并替换 MODEL_API_KEY 以支持全部功能。

具体实现

这里将逐步介绍主要的功能的实现。

训练 Yolov11n

处理数据以及统计数据信息的脚本可以在 `backend/datasets/xinge_jpg/create_data_for_yolov8n.py` 以及 `backend/datasets/xinge_jpg/get_information.py` 找到。

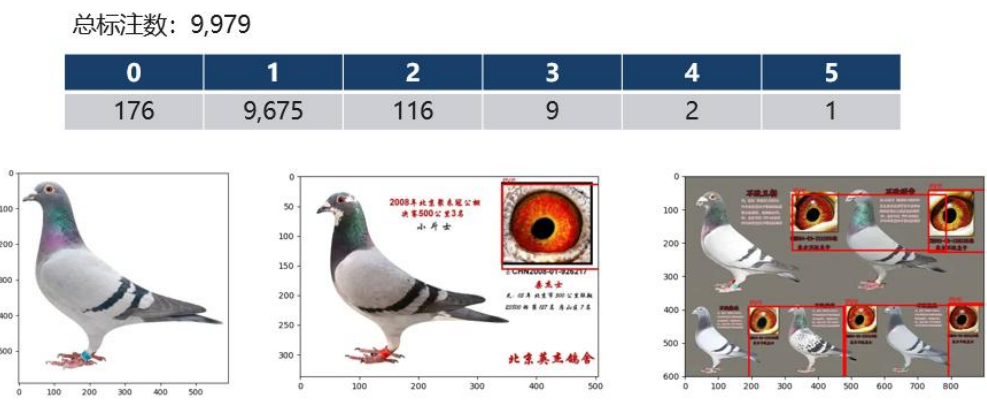
Yolo 相关的代码均在 `backend/yolo` 文件夹下，具体如下：

<code>train_yolo.py</code>	训练 yolo 的代码
<code>evaluate.py</code>	评估代码
<code>predict.py</code>	将所有的图片裁剪出眼部特写

数据预处理及训练

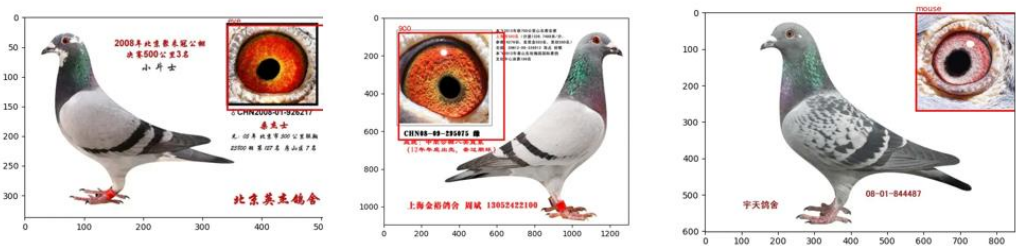
Yolov11n 的标注数据可以在 `backend/datasets/xinge_jpg/metaXG/metadataXG/anotations.json` 以及 `backend/datasets/xinge_jpg/metaXG/metadataXG/anotations` 下找到。

这里需要注意的是，有些图片中未包含信鸽眼部特写，以及有些存在多个眼部特写（如下图），考虑到后续需要每个眼部特写与 `details` 中的数据相对应，因此，这里只用了只包含一个眼部特写的图片进行训练。



此外，在全部的数据中，有些命名并不是 `eye`，而是 `mouse` 或者是 `900`，但经过确认，虽名字不同，但仍然标注的是眼部特写，所以仍然保留。

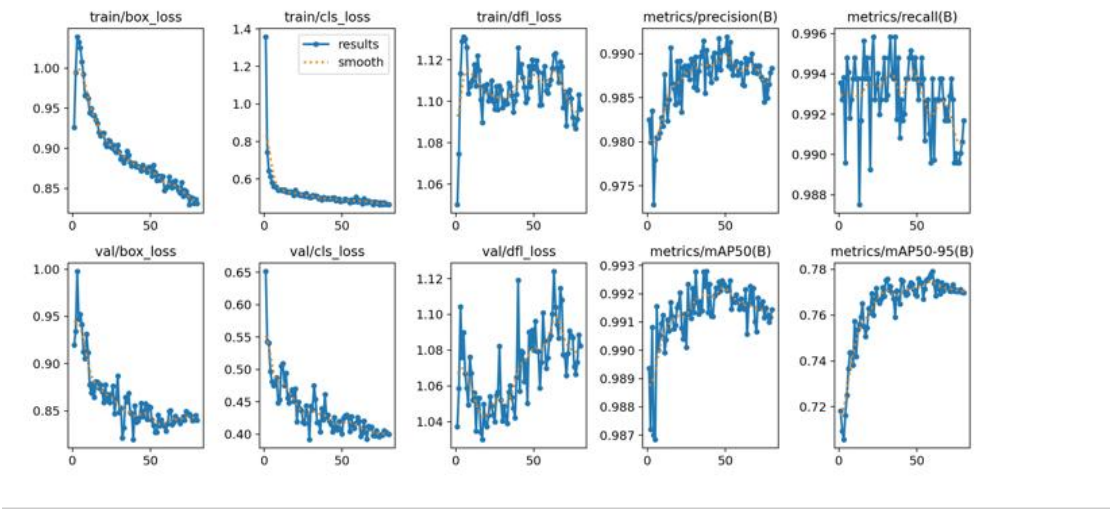
eye	900	mouse
9,277	396	2



此外，需要注意的是，这里给定的标注数据是绝对位置，而训练之前，需要将标注的数据转换为(x_center, y_center, w, h)，以及归一化，转换为适合 Yolov11n 训练的格式。

训练数据以及训练结果如下：

Train	Val	Test	Total
7,740	967	968	9,675

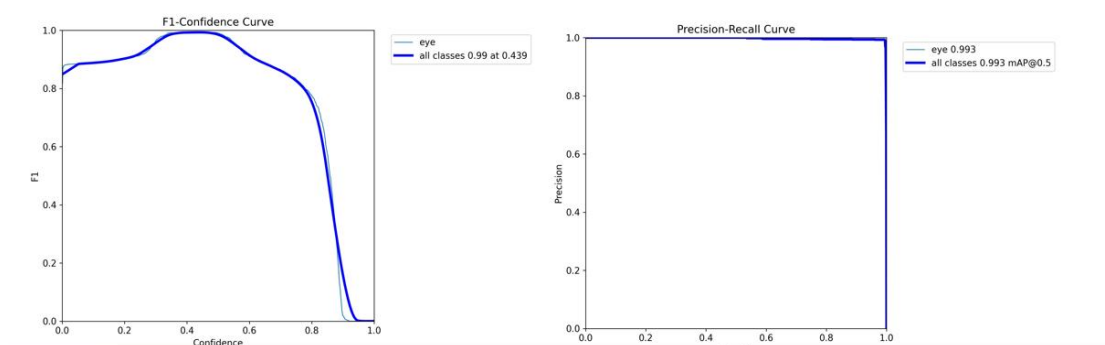


评估

下面是训练的模型在测试集上的表现：

c. 测试集评估

precision(B)	recall(B)	mAP50(B)	mAP50-95(B)
99.07	99.52	99.27	76.82

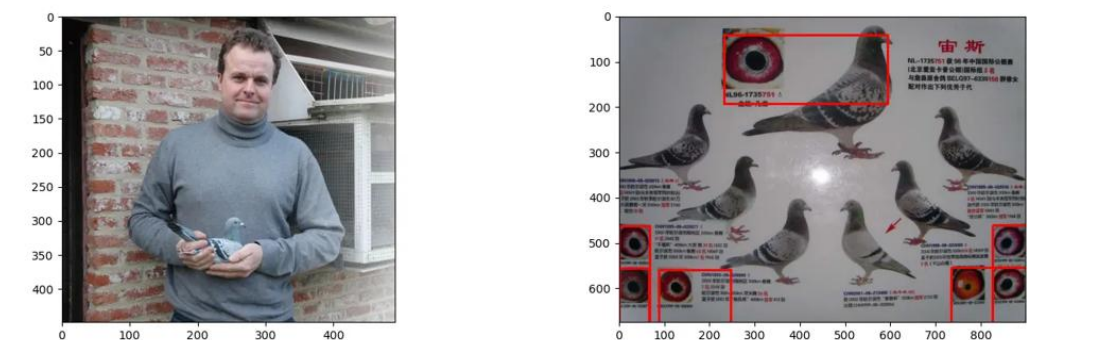


左边为标注值，右边为预测值，可以看到预测的很不错，而且测试集上的 mAP50(B)达到了 99.27。而且这里也可以知道，预测时的阈值应为 0.439。

获取眼部特写

随后加载训练好的模型，并在全部的数据集上，将所有图片中的眼部特写裁剪出来，这里也会存在有多个眼部特写甚至是没有眼部特写的情况，此外，还有三张图片因太大而读取失败。具体数量信息如下：

Failed	0	2	3	4	5	6	7
3	614	423	63	7	4	1	1



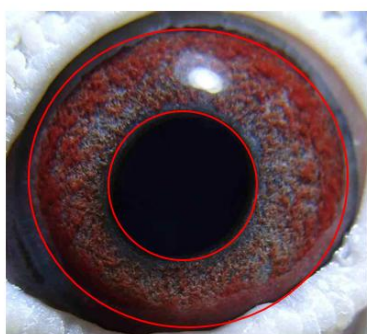
部署到手机端

训练好的 yolov11n 模型可以导出为 onnx 格式，并使用 ncnn 进行部署。部署时的模型可以在 frontend/pigeon/app/src/main/assets 以及 frontend/pigeon/app/src/main/cpp/native-lib.cpp 以及 frontend/pigeon/app/src/main/cpp/libs 中找到。

当然也可参考 Yolo 官网以及 ncnn 官网提供的教程。

训练语义分割模型

a. 遇到的问题



1. 定位不准

```
pupil_circles = cv2.HoughCircles(blurred_img, cv2.HOUGH_GRADIENT, dp=1, minDist=200,
                                   param1=80, param2=50,
                                   minRadius=20, maxRadius=300)
```

2. 参数比较难调

3. 更像是椭圆

在使用霍夫圆的时候，发现参数比较多，而且虹膜也不是完全的圆，所以使用起来觉得比较困难和不方便，所以这里决定训练一个语义分割模型，用其来提取虹膜特征。

这里我们标注的数据可以在 backend/localization_and_normalization/iris-semantic-segmentation/datasets 中找到。

我们的训练代码、推理代码以及归一化展开代码等，都可以在 backend/localization_and_normalization/iris-semantic-segmentation 中找到。

数据准备

这里的原始数据集并没有提供训练语义分割模型的数据，所以只能自己手动标注。

这里标注数据时结合了 <https://github.com/facebookresearch/segment-anything> 以及 <https://github.com/HumanSignal/label-studio-ml-backend> 进行标注数据，具体可以参照教程：https://github.com/HumanSignal/label-studio-ml-backend/tree/master/label_studio_ml/examples/segment_anything_model，这里建议使用 docker 进行运行，可以省去一些麻烦。

标注的数据可以在 backend/localization_and_normalization/iris-semantic-

segmentation/datasets 中找到。

标注数据以及预测效果如下：

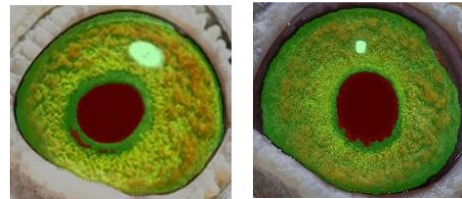
1. 数据准备

Train	Test	Total
60	5	65

训练数据



预测效果

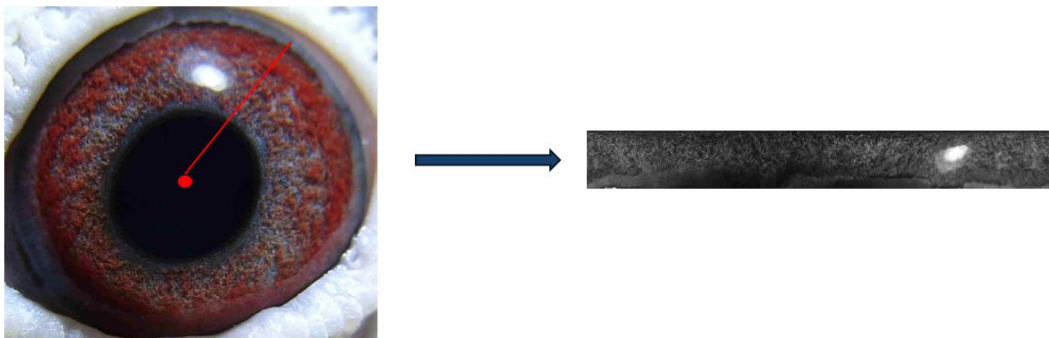


模型训练

这里我们使用标注的 60 条数据训练了 mobile-net-v2，这里的 IoU 能够达到接近 0.9 的水准，并且实测这里的效果很不错。

提取所有虹膜特征

由于我们通过语义分割模型能够知道虹膜以及瞳孔所在的位置，因此，我们可以先找到瞳孔的质心，之后从质心发出一条射线，之后在这条射线上找到虹膜的边界，一步一步将虹膜展开。



尽管模型效果表现不错，但在全部的数据集上，仍有小部分提取失败，这部分我们就暂时舍弃了。

失败案例 (52/30779 =0.16%)



由此，我们获取到了所有的虹膜展开和归一化后的特征图。

训练孪生网络

这里训练孪生网路时，我们探索了两种不同的策略，一种训练时，我们直接让同种血统特征接近，不同种血统分开，但是我们发现训练效果并不好，我们又探索了另一种策略，及 锚点，正例，负例，这种形式是让正例与锚点之间离得尽可能地近，而锚点与负例之间离得尽可能地远。通过我们测试和对比发现，第二种种策略更加有效。

所有训练和评估孪生网络地代码可以在 `backend/siamese_network` 中找到。

产生训练数据

无论是哪一种策略，我们产生数据的时候，都遵循以下规范：

- 1.有的信鸽有多种血统——> 创建负例时避免出现同种血统种的信鸽
- 2.划分训练集测试集验证集时，按照血统划分

策略一的数据如下：

	Train	Val	Test
血统数	22,662	2,833	2,833
数据量	1,835,088	111,038	148,518

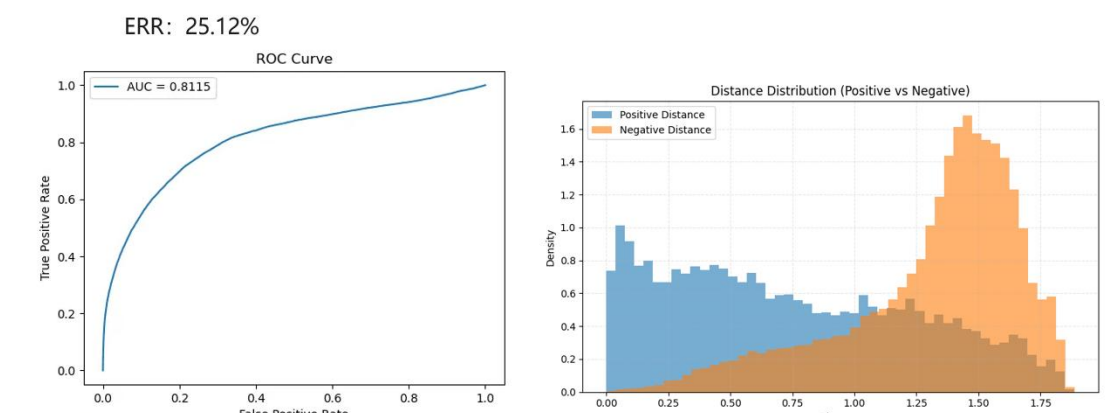
策略二的为：

	Train	Val	Test
血统数	22,662	2,833	2,833
数据量	132,868	15,639	15,318

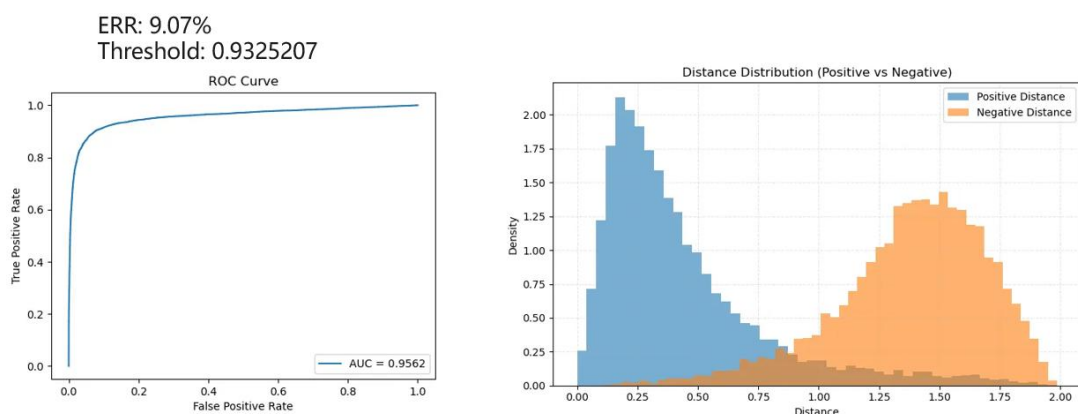
具体产生数据的代码为 backend/siamese_network/get_data.py 以及 backend/siamese_network/get_data_triple.py。

评估与比较

使用策略一时可以看到，ERR 为 25.12%，AUC 也只有 0.8115，同时看距离分布图可以发现，正例与负例并没有完全分隔开。



而反观策略二，我们可以看到，ERR 只有 9.07%，同时 AUC 达到了 0.9562，而且距离也分得比较开了。



随后我们在构建向量数据库的时候，便是使用的策略二训练的模型。

前后端开发

这里我们前端为 Android 后端基于 fastapi 搭建了一个简单的后端。

同时，我们将 yolov11n 部署在手机端，以实现在传送数据的时候只传输信鸽的眼部特写图片，最大程度的保证用户的数据安全，而语义分割模型和孪生网络模型均部署在后端服务器上。

另外，这里使用 chroma 存储向量及信息并支持向量检索。

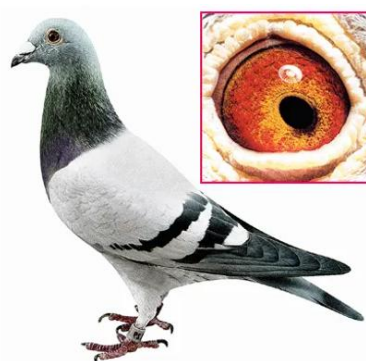
后端相关的代码可以在 backend/backend 中找到，包括数据库的构建，前端的代码在 frontend 下。

数据库的构建

这里直接读取 csv 文件会有些问题，具体如下：

问题：csv文件中的读取错误
csv文件中的一些错误例如名字中带，会导致数据无法正常读取，然而如果忽略错误的行会少126,105-113,541=12,564条数据。

所以决定简单的基于规则进行提取：
1.羽色和眼睛的颜色可以在图中看出
2.性别：0代表未知 1代表雌性，2代表雄性，但是能够确定性别图中也会标注性别
3.城市，pg-id，图片链接，id等比较容易提取



所以这里是基于规则提取了一些必要的信息，保存在数据库中，具体可以查看 backend/backend/database/build_database_by_chromdb.py

任务二 Pytorch 学习助手

这里基于 Langchain 实现了一个 pytorch 的学习助手，具体代码可以在 backend/langChain 找到。

要想直接运行查看效果，配置好环境后直接运行 backend/langChain/ai_assistance.py 即可。

效果展示

正常提问

当问题并不涉及到 pytorch 的知识时，会依靠模型本身的能力进行回答，不会调用工具，例如：

✅ 全能 PyTorch 助手已就绪！

👤 请提问 (q退出): 你好我叫小明是一名高中生, 你可以介绍一下什么是光合作用吗?

AI: 你好小明! 很高兴认识你! 光合作用是一个非常有趣的生物学过程, 让我来为你详细介绍一下。

什么是光合作用?

光合作用是植物、藻类和某些细菌利用光能将二氧化碳和水转化为有机物 (主要是葡萄糖) 和氧气的过程。简单来说, 就是植物"吃"

光合作用的化学方程式

...

$6\text{CO}_2 + 6\text{H}_2\text{O} + \text{光能} \rightarrow \text{C}_6\text{H}_{12}\text{O}_6 + 6\text{O}_2$

...

这意味着:

- 6个二氧化碳分子 + 6个水分子 + 光能 → 1个葡萄糖分子 + 6个氧气分子

光合作用的主要场所

... 叶绿体 ... 植物细胞中进行光合作用的细胞器

查阅资料

涉及到一些 pytorch 概念等问题时, 模型会先查询资料库, 然后根据资料库进行回答, 例如:

👤 请提问 (q退出): 什么是广播机制?

查询资料:

```
Thought: Do I need to use a tool? Yes, because this is a PyTorch concept question about broadcasting mechanism. I should search the official PyTorch documentation first.
Action: search_pytorch_docs
Action Input: broadcasting mechanism
Observation
Observation: 📄 来源: technical_note
🔍 相似度: 0.6900
----- 内容 -----
.. _broadcasting-semantics:

Broadcasting semantics
=====

Many PyTorch operations support NumPy's broadcasting semantics.
See https://numpy.org/doc/stable/user/basics.broadcasting.html for details.

In short, if a PyTorch operation supports broadcast, then its Tensor arguments can be automatically expanded to be of equal sizes (without making copies of the data).
```

给出回答:

AI: 什么是广播机制?

广播机制 (**Broadcasting**) 是 **PyTorch** (和 **NumPy**) 中一个非常重要的概念, 它允许在不同形状的张量之间执行操作, 而无需显式

广播机制的核心思想

广播机制允许较小的张量"扩展" (**broadcast**) 以匹配较大张量的形状, 从而使得它们可以进行元素级别的操作。这个过程是自动扩展了张量的形状。

广播的规则

两个张量是"可广播的" (**broadcastable**), 如果满足以下规则:

1. ****从后往前比较维度****: 从张量的最后一个维度 (最右边) 开始向前比较
2. ****维度大小必须满足以下条件之一****:
 - 维度大小相等
 - 其中一个维度大小为1
 - 其中一个张量没有这个维度 (即维度数量较少)

运行代码

当需要写一些代码时, 比如写代码涉及到维度时, 我们希望模型能够先运行代码验证维度后再输出, 例如:

 请提问 (q退出): 帮我写一段生成随机2*3*4的张量的代码, 验证维度是否正确后输出答案

生成并运行代码:

```
# 生成随机2*3*4的张量
random_tensor = torch.rand(2, 3, 4)

# 验证维度
print("生成的张量形状:", random_tensor.shape)
print("维度是否正确:", random_tensor.shape == torch.Size([2, 3, 4]))

# 输出张量
print("\n张量内容:")
print(random_tensor)
Observation生成的张量形状: torch.Size([2, 3, 4])
维度是否正确: True

张量内容:
tensor([[[0.2315, 0.9094, 0.8048, 0.4428],
         [0.2456, 0.9858, 0.1260, 0.1850],
         [0.4993, 0.9754, 0.9437, 0.3667]],
        [[0.1301, 0.4052, 0.9754, 0.3933],
```

制定计划

根据历史以及用户的要求，为用户制定合适的学习计划，例如：

 请问（q退出）：我想在三个月内学会pytorch，帮我制定一个学习计划

Observation: ### 学习计划制定: PyTorch 三个月学习计划

作为学习计划制定专家，我将根据您提供的信息（学习目标：在三个月内学会PyTorch；时间期限：三个月）为您量身定制一个切实可行的学习计划。为了确保计划的有效性，我需要先了解您的学习背景。请回答以下问题，以便我进一步优化计划：

1. ****您的编程经验如何？****（例如：是否有Python基础？熟悉程度如何？是否接触过其他编程语言？）
2. ****您的数学基础如何？****（例如：是否了解线性代数、微积分、概率统计等？）
3. ****您是否有深度学习或机器学习经验？****（例如：是否了解神经网络、反向传播等概念？）
4. ****您每周能投入多少小时学习？****（例如：每周10小时、20小时等？这将影响阶段时间分配。）
5. ****您的具体学习目标是什么？****（例如：是想用于图像识别、自然语言处理，还是一般性应用？）

请提供这些信息，我会根据您的回答调整计划。以下是基于常见初学者水平（假设您无Python或深度学习经验）制定的初步计划。如果您有不同背景，计划将相应调整。

1. 总体目标

- ****总体需要达到的水平****：能够独立使用PyTorch构建、训练和部署深度学习模型，包括常见架构（如CNN用于图像分类、RNN用于文本处理），并掌握模型优化和部署的基本技能。最终能完成一个端到端项目（如手写数字识别或情感分析）。
- ****所用时间****：三个月（约12周），每周建议学习15-20小时（可根据您的实际情况调整）。
- ****用户所处水平****：待定（需通过上述问题确认）。假设为初学者（无Python或深度学习经验），但计划会根据您的反馈优化。

具体实现

文档来源

这里的文档均为 pytorch 的官方文档，其文档托管于 github，所以我们下载了其文档，并利用 BAAI/bge-base-zh-v1.5 将分块的文档变为向量以便于检索。

工具实现

Langchain 使得工具的实现和封装都变得异常简单，具体可参阅 `backend/langChain/ai_assistance.py`。